



Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

„ VÍZCSEPP MOSÁS”

Készítették:

Galambosi Dániel,
Megyery Bese Máté,
Kiss Bálint

Ózd, 2026.03.30

Tartalom

1. Szükséges szoftverek a webalkalmazáshoz	5
2. A webalkalmazás indítása.....	5
2.1. Backend indítása.....	5
2.2. Frontend indítása.....	5
3. Alkalmazott fejlesztői és csoportmunka eszközök	6
3.1. Tervezés és projektmenedzsment	6
3.2. Fejlesztői környezet	6
3.3. Verziókezelés	6
3.4. Frontend technológiák	6
3.5. Backend technológiák.....	7
3.6. Szerver környezet	7
4. Adatbázis	7
5. Frontend	7
6. Backend	8
7. Rendszerkövetelmények	8
7.1. Szerveroldali követelmények.....	8
7.2. Kliensoldali követelmények	8
7.3. Minimális hardverigény	9
Fontos metódusok.....	14
Validálások és ellenőrzések.....	14
Visszatérési értékek	14
HTTP státuszkódok	14
1. A projekt bemutatása.....	23
2. Felhasznált technológiák	23
3. Az alkalmazás felépítése.....	23
4. Regisztrációs felület.....	24
5. Bejelentkezési felület	25
6. Felhasználói felület.....	28
7. Angular működés a projektben	28
8. Frontend működés	28
9. Komponensek bemutatása.....	29
Home	29
Login	29
Register.....	29
Account.....	29
Admin	29
10. Backend működés	30
11. API működés.....	30

12. Adatbázis működés	31
13. Jogosultságkezelés	31
14. Admin felület	31
15. Megrendelések kezelése	33
16. Felhasználói adatok kezelése	34
17. Adatbiztonság.....	36
18. Hibakezelés.....	36
19. Tesztelés	36
20. Felhasználói élmény	37
21. Reszponzív design.....	37
22. Rendszer előnyei	37
23. Továbbfejlesztési lehetőségek	37
24. Összegzés.....	37

Bevezetés:

Ma már egyre többen értékelik, ha az autójuk tiszta és ápolat. Egy tiszta autó nemcsak szép, de megőrzi az értékét és tovább használható jó állapotban. Az ember élete felgyorsult, ezért sokaknak nincs idejük saját maguknak kitakarítani az autójukat. Ezért keresnek megbízható, gyors és kényelmes autótakarítási szolgáltatásokat. Projektünk célja, hogy bemutassa, milyen sokféle lehetőség van az autók tisztítására, és hogyan segíthet ez a szolgáltatás mindennapi életünkben. A projektben részletesen bemutatjuk az autótakarítás lépéseit, a különböző tisztítási módszereket és technológiákat, például a kézi mosást, a gépi autómosót és a mobil autómosó szolgáltatásokat is. Fontosnak tartjuk megemlíteni a környezetbarát megoldásokat, amelyek egyre népszerűbbek, hiszen kevesebb vizet használnak és egészségesebbek a természet számára. Az autótakarítás nem csak kívülről, hanem belülről is fontos: a kárpitok, ülések, műszerfal tisztítása ugyanolyan figyelmet érdemel, hogy az autó minden része higiénikus és komfortos legyen. Projektünk arra is kitér, hogy az autótisztítás üzleti szempontból is jó lehetőség. Egy jól szervezett autótakarító vállalkozás most különösen keresett, mert az emberek sokszor nem tudják vagy nem akarják saját maguk elvégezni a feladatot. Bemutatjuk, milyen lépések szükségesek egy autótakarító cég elindításához, milyen technológiákat használhatnak, és milyen piaci igények vannak ma Magyarországon. A cél, hogy aki ezen a területen vállalkozni szeretne, könnyen eligazodjon a lehetőségek között. Végül arra is hangsúlyt fektetünk, hogy az autótakarítás nem csak az autó tisztaságáról szól, hanem az autótulajdonosok kényelméről, elégedettségéről és a környezetvédelemről is. Egy tiszta, jól karbantartott autó hozzájárul a biztonsághoz és a jobb közlekedési élményhez. Ezzel a projekttel szeretnénk megmutatni, hogy az autótakarítás egy sokszínű és fejlődő terület, amely mindenki számára előnyös lehet: az autótulajdonosok számára időt és energiát spórol, a vállalkozók számára pedig biztonságos és jövedelmező üzleti lehetőséget kínál.

1. Szükséges szoftverek a webalkalmazáshoz

A webalkalmazás megfelelő működéséhez és futtatásához több szoftver telepítése szükséges. Ezek biztosítják a backend és frontend környezet működését, valamint a fejlesztési folyamat támogatását.

A szükséges szoftverek:

- **XAMPP (Apache + MySQL)** – a helyi webservert és adatbázis futtatásához
- **PHP** – a backend működéséhez (a XAMPP részeként települ)
- **Composer** – a Laravel backend függőségeinek telepítéséhez
- **Node.js és npm** – az Angular frontend futtatásához
- **Angular CLI** – az Angular alkalmazás fejlesztéséhez és futtatásához
- **Visual Studio Code** – a projekt szerkesztéséhez
- **Böngésző (Chrome, Edge, Firefox)** – az alkalmazás megjelenítéséhez

Ezek a szoftverek együtt biztosítják a teljes fejlesztési és futtatási környezetet.

2. A webalkalmazás indítása

2.1. Backend indítása

A backend indításához az alábbi lépések szükségesek:

1. Indítsa el a **XAMPP Control Panelt**
2. Indítsa el az **Apache** és **MySQL** szolgáltatásokat
3. Nyissa meg a backend projekt mappáját **Visual Studio Code**-ban
4. Terminálban futtassa az alábbi parancsokat:

```
composer install
php artisan migrate --seed
php artisan serve
```

A backend ezek után az alábbi címen lesz elérhető:

2.2. Frontend indítása

A frontend futtatásához:

1. Nyissa meg a frontend projekt mappáját
2. Terminálban futtassa:

```
npm install
ng serve
```

3. Alkalmazott fejlesztői és csoportmunka eszközök

A projekt során több különböző eszközt használtunk a hatékony fejlesztés és együttműködés érdekében.

3.1. Tervezés és projektmenedzsment

Figma

A felhasználói felület megtervezéséhez a Figma online tervezőprogramot használtuk. Segítségével előre elkészítettük a design terveket, így a fejlesztés során pontosan tudtuk követni a kívánt megjelenést.

Trello

A feladatok nyomon követésére és szervezésére a Trello alkalmazást használtuk. A feladatokat listákba és kártyákba rendeztük, így a csapat minden tagja átláthatta a munkafolyamatokat.

3.2. Fejlesztői környezet

Visual Studio Code (v1.96 vagy újabb)

A fejlesztés során a Visual Studio Code szerkesztőt használtuk, amely támogatja a HTML, CSS, JavaScript, Angular és PHP nyelveket. Kiegészítő bővítményekkel gyorsabb és hatékonyabb fejlesztést tett lehetővé.

3.3. Verziókezelés

GitHub

A projekt verziókezelésére a GitHub platformot használtuk. Segítségével a forráskód változásai követhetők voltak, valamint lehetővé tette a csapat tagjai közötti együttműködést.

3.4. Frontend technológiák

Angular (v17 vagy újabb)

A webalkalmazás kliensoldali részének fejlesztéséhez az Angular keretrendszert használtuk, amely komponens-alapú felépítést biztosít.

Node.js (v22 vagy újabb)

Az Angular alkalmazás futtatásához és a csomagkezeléshez Node.js környezetet használtunk.

3.5. Backend technológiák

Laravel (v10 vagy újabb)

A backend fejlesztéséhez a Laravel PHP keretrendszert használtuk, amely lehetővé teszi API-k létrehozását és az adatbázis kezelését.

PHP (v8.1 vagy újabb)

A backend működéséhez szükséges programozási nyelv.

MySQL (v8 vagy újabb)

Az alkalmazás adatainak tárolására használt adatbázis-kezelő rendszer.

3.6. Szerver környezet

XAMPP (v8.1 vagy újabb)

A helyi fejlesztéshez használt környezet, amely tartalmazza az Apache webszervert, a MySQL adatbázist és a PHP futtatókörnyezetet.

4. Adatbázis

Az alkalmazás MySQL adatbázist használ az adatok tárolására.

A tárolt adatok:

- Felhasználók (név, email, telefonszám)
- Autók (márka, modell, évjárat, rendszám)

Az adatbázis kezelése:

- Laravel migrációk segítségével
- phpMyAdmin felületen keresztül

Ez biztosítja az adatok strukturált és biztonságos kezelését.

5. Frontend

A frontend Angular keretrendszerrel készült.

Feladata:

- felhasználói felület megjelenítése
- adatok bekérése (login, register)
- backend API hívása

Fő funkciók:

- bejelentkezés és regisztráció
 - főoldal (szolgáltatások, árak)
 - felhasználói fiók
 - autók kezelése
-

6. Backend

A backend Laravel keretrendszerrel készült.

Feladata:

- API biztosítása
- adatbázis kezelés
- felhasználók kezelése

Fő funkciók:

- regisztráció
 - bejelentkezés
 - autók kezelése
 - megrendelések kezelése
-

7. Rendszerkövetelmények

7.1. Szerveroldali követelmények

- Operációs rendszer: Windows vagy Linux
 - Backend: PHP (Laravel)
 - Adatbázis: MySQL
 - RAM: minimum 4 GB
-

7.2. Kliensoldali követelmények

- Böngésző: Chrome, Firefox, Edge
 - Internetkapcsolat szükséges
-

7.3. Minimális hardverigény

- Legalább 2 magos processzor
- Minimum 4 GB RAM
- Stabil internetkapcsolat

Adatbázis dokumentáció:

I. Áttekintés

Az adatbázis egy autómosó szolgáltatás ügyfél-, jármű-, munka- és szolgáltatáskezelő modulját valósítja meg. A rendszerben felhasználók rendelkeznek autókkal, a takarítók elvégzik a munkákat, és minden munkához hozzá tartozik egy szolgáltatás, amelynek ára rögzítésre kerül.

II. Adatbázis szerkezete

Az adatbázis öt táblából áll:

1. **Felhasznalo** – autótulajdonosok adatai
2. **Auto** – felhasználókhoz tartozó járművek
3. **Takarito** – munkát végző dolgozók
4. **Munka** – elvégzett tisztítási munkák
5. **Szolgáltatás** – munkák és árak összekötése(revízió előtt áll)

Az adatbázis idegen kulcsokon keresztül kapcsolódik össze, biztosítva az adatok konzisztenciáját.

III. Táblák részletes leírása

III.1. Felhasznalo tábla

A felhasználókat (ügyfeleket) tárolja.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Egyedi felhasználói azonosító
nev	VARCHAR(100), NOT NULL	Felhasználó neve
phonenumber	VARCHAR(30)	Telefonszám
email	VARCHAR(100)	Email cím

Kapcsolatok:

- 1:N kapcsolat az **Auto** táblával (egy felhasználónak több autója lehet).
 - 1:N kapcsolat a **Munka** táblával (egy felhasználó több munkát rendelhet).
-

III.2. Auto tábla

Az ügyfelekhez tartozó autókat rögzíti.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Egyedi autóazonosító
marka	VARCHAR(50)	Autó márkája
tipus	VARCHAR(50)	Autó típusa

Mező	Típus	Leírás
evjarat	INT	Gyártási év
rendszam	VARCHAR(20), UNIQUE	Rendszám
tulaj	INT, FK → Felhasznalo(ID)	Az autó tulajdonosa

Kapcsolatok:

- N:1 a **Felhasznalo** táblával
- 1:N a **Munka** táblával

III.3. Takarito tábla

Az autómosó takarítóinak adatait tartalmazza.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Takarító azonosító
nev	VARCHAR(100), NOT NULL	Név
phonenumber	VARCHAR(30)	Telefonszám

Kapcsolatok:

- 1:N a **Munka** táblával (egy takarító több munkát végezhet)

III.4. Munka tábla

A ténylegesen elvégzett munkákat tartalmazza. Minden munkához tartozik egy autó, egy felhasználó és opcionálisan egy takarító.

Mező	Típus	Leírás
ID	INT, PK, AUTO_INCREMENT	Munka azonosító
auto	INT, NOT NULL, FK → Auto(ID)	A tisztított autó
felhasznalo	INT, NOT NULL, FK → Felhasznalo(ID)	Megrendelő
takarito	INT, FK → Takarito(ID)	A munkát végző takarító
datum	DATE	Munka dátuma

Kapcsolatok:

- N:1 az **Auto** táblával
- N:1 a **Felhasznalo** táblával
- N:1 a **Takarito** táblával

- 1:N a **Szolgáltatás** táblával

III.5. Szolgáltatás tábla

A munkához tartozó szolgáltatás(ok) árát rögzíti.

Mező Típus

Leírás

ID INT, PK, AUTO_INCREMENT

Szolgáltatás azonosító

munka INT, NOT NULL, FK → Munka(ID)

Melyik munkához tartozik

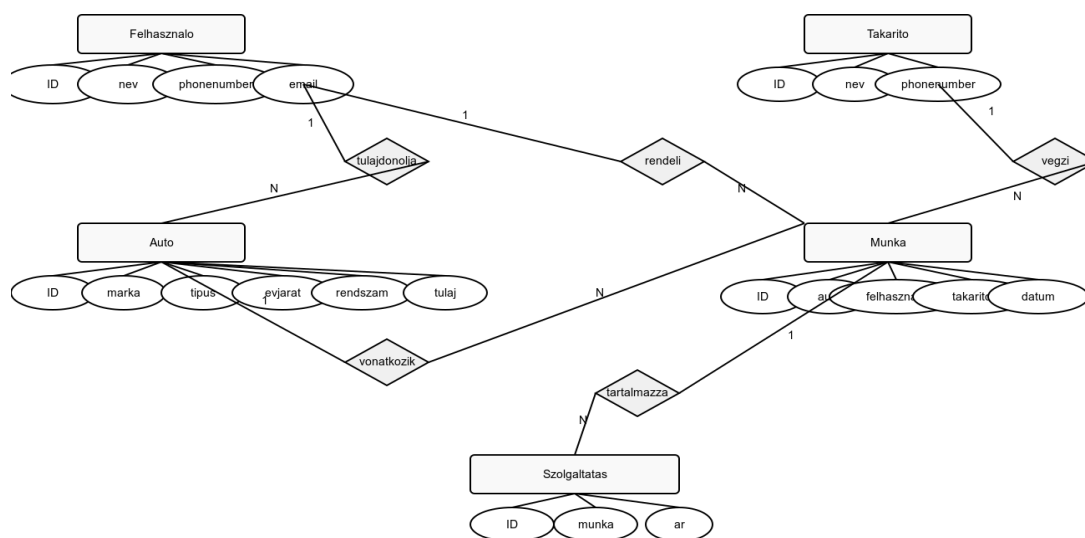
ar INT, NOT NULL

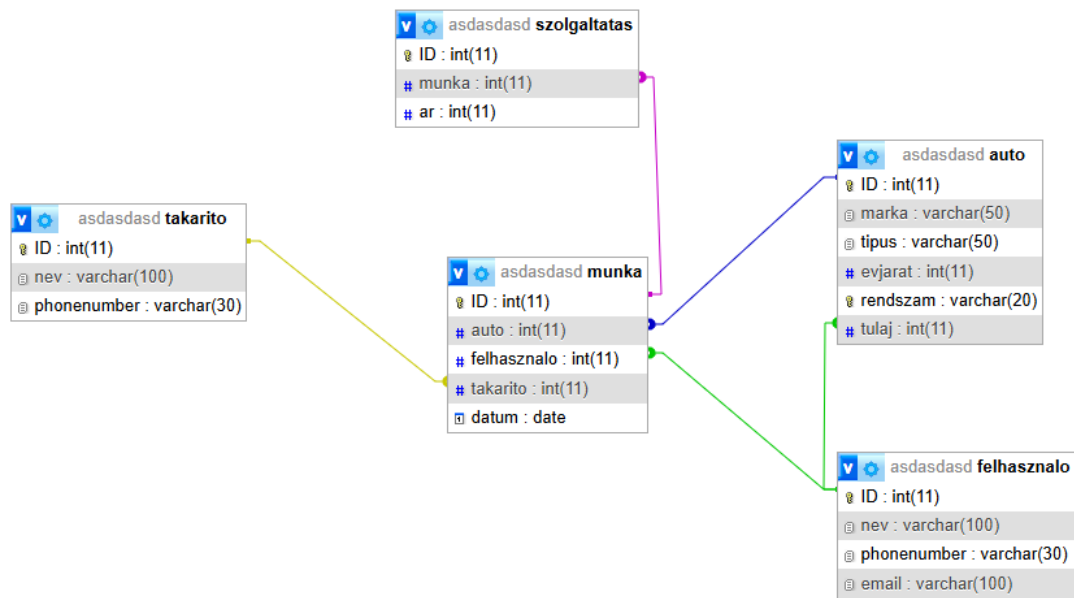
Szolgáltatás ára (HUF)

Kapcsolatok:

- N:1 a **Munka** táblával

IV. E-K Diagram





Backend dokumentáció:

Fontos metódusok

- **store** – új autó létrehozása
- **update** – meglévő autó frissítése
- **getByOwner** – autók lekérdezése felhasználó alapján

Validálások és ellenőrzések

- Kötelező mezők meglétének ellenőrzése
- Autó létezésének ellenőrzése frissítés előtt

Visszatérési értékek

- JSON formátumú válaszok
- Sikeres és hibás esetek elkülönítése

HTTP státuszkódok

- **201** – sikeres létrehozás
- **206** – sikeres frissítés
- **404** – nem található erőforrás
- **406** – hibás vagy hiányos adatok

```

public function store(Request $request)
{
    $validator=Validator::make($request->all(),
    [
        'marka'=>'required',
        'tipus'=>'required',
        'evjarat'=>'required',
        'rendszam' => 'required',
        'felhasznalo' => 'required'
    ]);

    if($validator->fails())
    {
        return response()->json(['Hibaüzenet'=>'Valamelyik kötelező adat hiányzik'],406);
    }

    $auto=Auto::create($request->all());
    return response()->json(['Új autó létrehozva. Rendszám:'=>$auto->rendszam],201);
}

```

A **store()** metódus új autó rögzítéséért felelős. A metódus egy HTTP kérésből (**Request**) kapja meg az adatokat, amelyek tartalmazzák az autó márkáját, típusát, évjáratát, rendszámát és a hozzá tartozó felhasználót. Ezek az adatok kötelezőek, ezért a metódus validációt végez annak ellenőrzésére, hogy minden szükséges mező kitöltésre került-e. Amennyiben valamelyik adat hiányzik, a rendszer egy JSON formátumú hibaválaszt ad vissza „Valamelyik kötelező adat hiányzik” üzenettel, 406-os HTTP státuszkóddal. Ha a validáció sikeres, az autó adatai elmentésre kerülnek az adatbázisba, és a rendszer 201-es státuszkóddal jelzi az erőforrás sikeres létrehozását, valamint visszaküldi az újonnan létrehozott autó rendszámát.

```

public function getByOwner(Request $request){
    $auto = Auto::where('felhasznalo', '=', $request -> felhasznalo);
    if($auto -> exists()){
        return $auto -> get();
    }
    else{
        return response() -> json(['Hiba:' => 'Nincs regisztrálva autó ezzel a rendszámmal.'], 404);
    }
}

public function update(Request $request, $id){

```

A **getByOwner()** metódus egy adott felhasználóhoz tartozó autók lekérdezésére szolgál. A metódus a kérésben kapott felhasználónév alapján keres az adatbázisban. Először ellenőrzi, hogy létezik-e legalább egy autó, amely ehhez a felhasználóhoz van rendelve. Amennyiben található ilyen bejegyzés, a rendszer visszaadja az autók listáját JSON formátumban. Ha nem található egyetlen autó sem, akkor a rendszer 404-es HTTP státuszkóddal tér vissza, és egy hibaüzenetet küld, amely jelzi, hogy nincs regisztrált autó az adott feltételekkel.

```

public function update(Request $request, $id){

    $auto = Auto::find($id);
    if(is_null($auto)){
        return response()->json(['Hiba:' => 'Nem található a frissíteni kívánt autó.'], 404);
    }
    else{

        $validator=Validator::make($request->all(),
        [
            'marka'=>'required',
            'típus'=>'required',
            'evjarat'=>'required',
            'rendszer' => 'required',
            'felhasznalo' => 'required'
        ]);

        if($validator->fails())
        {
            return response()->json(['Hibaüzenet'=>'Valamelyik kötelező adat hiányzik'],406);
        }
        else{
            $auto -> update($request);
            return response()-> json(['Autó sikeresen frissítve:' => $auto -> rendszer], 206);
        }
    }
}

```

Az **update()** metódus egy már meglévő autó adatainak frissítését végzi el az autó azonosítója alapján. A metódus első lépésként megvizsgálja, hogy az adott azonosítóval létezik-e autó az adatbázisban. Ha nem található ilyen rekord, a rendszer 404-es státuszkóddal és egy „Nem található a frissíteni kívánt autó” üzenettel válaszol. Amennyiben az autó létezik, a metódus ismét validálja a beérkező adatokat, mivel a frissítés során is minden mező kitöltése kötelező. Sikertelen validáció esetén a rendszer 406-os státuszkóddal jelez hibát. Ha minden adat megfelelő, az autó frissítésre kerül, és a rendszer 206-os HTTP státuszkóddal válaszol, amely jelzi a frissítés sikerességét, valamint visszaküldi az érintett autó rendszámát.

```

87
88 public function destroy($id){
89     $auto = Auto::find($id);
90     if(is_null($auto)){
91         return response('A törölni kívánt autó nem található.', 404);
92     }
93     else{
94         $auto -> delete();
95         return response('Autó eltávolítva az adatbázisból.', 202);
96     }
97 }
98
99
100

```

A **destroy(\$id)** metódus egy autó törlésére szolgál az adatbázisból egy megadott azonosító alapján. A metódus paraméterként megkapja az autó azonosítóját, majd megpróbálja megkeresni azt az adatbázisban. Ha az autó nem létezik, akkor egy 404-es HTTP státuszkóddal tér vissza, és azt az üzenetet adja vissza, hogy a törölni kívánt autó nem található. Amennyiben az autó létezik, a metódus törli azt az adatbázisból, majd 202-es státuszkóddal válaszol, „Autó eltávolítva az adatbázisból.” üzenettel. Ez a működés azért fontos, mert megakadályozza, hogy nem létező adatokat próbáljunk törölni, és egyértelmű visszajelzést ad a kliens számára.

```

62
63 public function searchByName(Request $request){
64     $felhasznalo = Felhasznalo::where('nev', '=', $request -> nev);
65     if($felhasznalo -> exists()){
66         return $felhasznalo -> get();
67     }
68     else{
69         return response() -> json(['Hiba:' => 'Nem található a keresésnek megfelelő felhasználó.'], 404);
70     }
71 }
72

```

A **searchByName(Request \$request)** metódus felhasználók keresését végzi név alapján. A metódus egy HTTP requestet kap paraméterként, amely tartalmazza a keresett felhasználó nevét. A controller az adatbázisban megvizsgálja, hogy létezik-e olyan felhasználó, akinek a neve megegyezik a megadott értékkel. Ha van találat, akkor a metódus visszaadja a megtalált felhasználó vagy felhasználók adatait, alapértelmezett 200-as HTTP státuszkóddal. Ha nincs találat, akkor 404-es státuszkóddal tér vissza, és egy JSON formátumú hibaüzenetet küld, amely jelzi, hogy nem található a keresésnek megfelelő felhasználó. Ez a metódus azért fontos, mert lehetőséget ad az adatok gyors visszakeresésére, miközben a hibás vagy eredménytelen kereséseket is megfelelően kezeli.

Backend metódusok tesztjei:

index()

Az összes objektumtípus **index()** metódusainak teszteredményei. A szerver az elérési út által megjelölt adattáblák összes elemét elküldi a kliens felé JSON formátumban. Abban az esetben, ha egy táblában idegen kulcs szerepel, a hozzá tartozó mező adatait a JSON válaszon belül **belongsTo()** szerkezet segítségével adja vissza. A metódus minden esetben **200**-as HTTP státuszkódot (OK) ad vissza, ezzel jelezve, hogy az adatlekérés sikeres.

- **index()** a takarítók táblán:

Query	Headers	Auth	Body	Tests	Pre Run	Status: 200 OK	Size: 2.49 KB	Time: 243 ms
GET			http://127.0.0.1:8000/api/takaritok					
JSON XML Text Form Form-encode GraphQL Binary						Response Headers Cookies Results Docs		
JSON Content <pre> 1 { 2 "nev": "" 3 } </pre>						<pre> 1 [2 { 3 "id": 1, 4 "nev": "Bíró Tamás", 5 "phoneNumber": "06709000001" 6 }, 7 { 8 "id": 2, 9 "nev": "Varga Csaba", 10 "phoneNumber": "06709000002" 11 }, 12 { 13 "id": 3, 14 "nev": "Tóth Zoltán", 15 "phoneNumber": "06709000003" 16 }, 17 { 18 "id": 4, 19 "nev": "Kiss Laura", 20 "phoneNumber": "06709000004" 21 }, 22 { 23 "id": 5, 24 "nev": "Molnár Bence", 25 "phoneNumber": "06709000005" 26 }, 27] </pre>		

- **index()** a szolgáltatások táblán:

Query	Headers	Auth	Body	Tests	Pre Run	Status: 200 OK	Size: 3.9 KB	Time: 200 ms
GET			http://127.0.0.1:8000/api/szolgo					
JSON XML Text Form Form-encode GraphQL Binary						Response Headers Cookies Results Docs		
JSON Content <pre> 1 { 2 "datum": "1848-03-15" 3 } </pre>						<pre> 1 [2 { 3 "id": 1, 4 "munka": { 5 "id": 1, 6 "auto": 1, 7 "felhasznalo": 1, 8 "takarito": 1, 9 "datum": "2026-01-04" 10 }, 11 "ar": 15000 12 }, 13 { 14 "id": 2, 15 "munka": { 16 "id": 2, 17 "auto": 2, 18 "felhasznalo": 2, 19 "takarito": 2, 20 "datum": "2026-01-07" 21 }, 22 "ar": 18000 23 }, 24 { 25 "id": 3, 26 "munka": { 27 "id": 3, </pre>		

- **index()** a munkak táblán:

GET

http://127.0.0.1:8000/api/munkak

Send

Query

Headers

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```

1 {
2
3
4   "nev": ""
5 }

```

Status: 200 OK Size: 12.54 KB Time: 194 ms

Response

Headers

Cookies

Results

Docs

```

1 [
2   {
3     "id": 1,
4     "auto": {
5       "id": 1,
6       "marka": "Audi",
7       "tipus": "A4",
8       "evjarat": 1994,
9       "rendszam": "AIIIR-435",
10      "felhasznalo": 1
11    },
12    "felhasznalo": {
13      "id": 1,
14      "nev": "Kovács István",
15      "phonenumber": "06201110001",
16      "email": "istvan.kovacs1@example.com"
17    },
18    "takarito": {
19      "id": 1,
20      "nev": "Bíró Tamás",
21      "phonenumber": "06700000001"
22    },
23    "datum": "2026-01-04"
24  },
25  {
26    "id": 2,
27    "auto": {

```

- **index()** a felhasználók táblán:

GET

http://127.0.0.1:8000/api/felhasznalok

Send

Query

Headers

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```

1 {
2
3
4   "nev": ""
5 }

```

Status: 200 OK Size: 3.92 KB Time: 189 ms

Response

Headers

Cookies

Results

Docs

```

1 [
2   {
3     "id": 1,
4     "nev": "Kovács István",
5     "phonenumber": "06201110001",
6     "email": "istvan.kovacs1@example.com"
7   },
8   {
9     "id": 2,
10    "nev": "Szabó Mária",
11    "phonenumber": "06201110002",
12    "email": "maria.szabo2@example.com"
13  },
14  {
15    "id": 3,
16    "nev": "Varga Lajos",
17    "phonenumber": "06201110003",
18    "email": "lajos.varga3@example.com"
19  },
20  {
21    "id": 4,
22    "nev": "Horváth Kitti",
23    "phonenumber": "06201110004",
24    "email": "kitti.horvath4@example.com"
25  },
26  {
27    "id": 5,

```

- **index()** az **autok** táblán:

The screenshot shows a REST client interface. The top bar displays the method **GET** and the URL **http://127.0.0.1:8000/api/autok**. The status bar indicates **Status: 200 OK**, **Size: 7.5 KB**, and **Time: 309 ms**. The left pane shows the **Query Parameters** section with a table:

parameter	value

The right pane shows the **Response** in JSON format:

```
[
  {
    "id": 1,
    "marka": "Toyota",
    "tipus": "Yaris",
    "evjarat": 2019,
    "rendszám": "AAA-101",
    "felhasználó": {
      "id": 1,
      "nev": "Kovács István",
      "phonenumber": "06201110001",
      "email": "istvan.kovacs1@example.com"
    }
  },
  {
    "id": 2,
    "marka": "Honda",
    "tipus": "Civic",
    "evjarat": 2017,
    "rendszám": "AAA-102",
    "felhasználó": {
      "id": 2,
      "nev": "Szabó Mária",
      "phonenumber": "06201110002",
      "email": "maria.szabo2@example.com"
    }
  }
]
```

searchByName()

A **getByName()** metódus két táblán használható(**felhasznalok**, **takaritok**), és 2 lehetséges státuszkódot adhat vissza:

- Egy egyed sikeres lekérésénél

POST ()

A **POST()** metódus 2 lehetséges státuszkódot adhat vissza:

- Egyed sikeres létrehozásakor **201**-es HTTP státuszkódot(létrehozva) ad vissza, illetve JSON formátumban egy létrehozást megerősítő üzenetet, ami a létrehozott autó rendszámát, felhasználó/takarító nevét, munka/szolgáltatás esetében dátumát tartalmazza.
- Sikeres **POST()** az **autok** táblán:

POST

http://127.0.0.1:8000/api/autok

Send

Query

Headers

Auth

Body

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

```
1 {
2
3   "marka": "Toyota",
4   "tipus": "Vanis",
5   "evjarat": 2019,
6   "rendszer": "AAX-101",
7   "felhasznalo": 1
8 }
```

Status: 201 Created

Size: 63 Bytes

Time: 266 ms

Response

Headers

Cookies

Results

Docs

```
1 {
2   "Új autó létrehozva. Rendszám:": "AAX-101"
3 }
```

- Ha az egyed létrehozása sikertelen, a metódus **406**-os státuszkódot(nem elfogadható) ad vissza, illetve egy JSON formátumú üzenetet, miszerint a feltöltendő egyed adatformátuma hibás.
- Sikertelen **POST()** az **autok** táblán:

POST http://127.0.0.1:8000/api/autok

Send

Status: 406 Not Acceptable Size: 70 Bytes Time: 237 ms

Query Headers Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```

1 {
2   "típus": "Vanis",
3   "evjarat": 2019,
4   "rendszam": "AAX-101",
5   "felhasznalo": 1
6 }

```

Response Headers Cookies Results Docs

```

1 {
2   "Hibaüzenet": "Valamelyik kötelező adat hiányzik"
3 }

```

1. A projekt bemutatása

A projekt célja egy modern, reszponzív webalkalmazás létrehozása, amely egy autómosó szolgáltatás online működését támogatja. Az alkalmazás lehetőséget biztosít a felhasználók számára, hogy regisztráljanak, bejelentkezzenek, valamint kezeljék saját autóikat és megrendeléseiket.

A rendszer Angular frontend és Laravel backend technológiák felhasználásával készült, amely biztosítja a gyors, stabil és jól strukturált működést. A felhasználói felület letisztult és könnyen kezelhető, így minden korosztály számára érthető és használható.

A projekt célja az volt, hogy bemutassa egy teljes webalkalmazás fejlesztésének folyamatát, beleértve a frontend és backend integrációt, adatbázis kezelést, valamint a felhasználói hitelesítést.

2. Felhasznált technológiák

A projekt megvalósításához az alábbi technológiákat használtuk:

- Angular – kliensoldali fejlesztés
- TypeScript – programozási nyelv
- HTML5 – szerkezet
- CSS / Bootstrap – megjelenítés
- Laravel – backend fejlesztés
- MySQL – adatbázis
- Node.js és Angular CLI – fejlesztési környezet

Az Angular CLI segítségével történt a projekt létrehozása, a komponensek generálása és az alkalmazás futtatása.

3. Az alkalmazás felépítése

Az alkalmazás komponens alapú struktúrában készült, amely lehetővé teszi az átlátható és könnyen karbantartható kódot.

Fő részek:

- Főoldal
- Regisztráció
- Bejelentkezés
- Felhasználói fiók
- Admin felületA navigáció menüsor segítségével történik.

4. Regisztrációs felület

A regisztrációs oldal új felhasználók létrehozását teszi lehetővé.

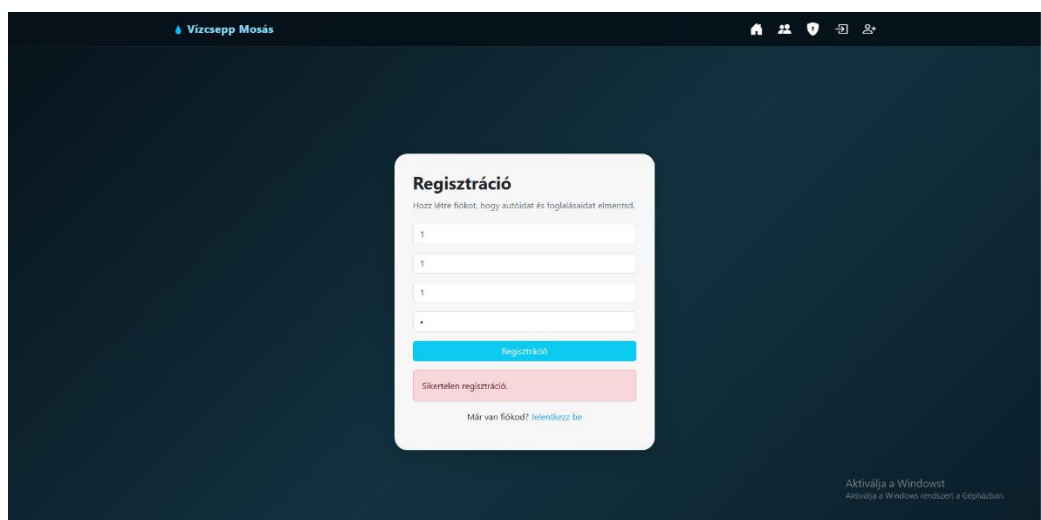
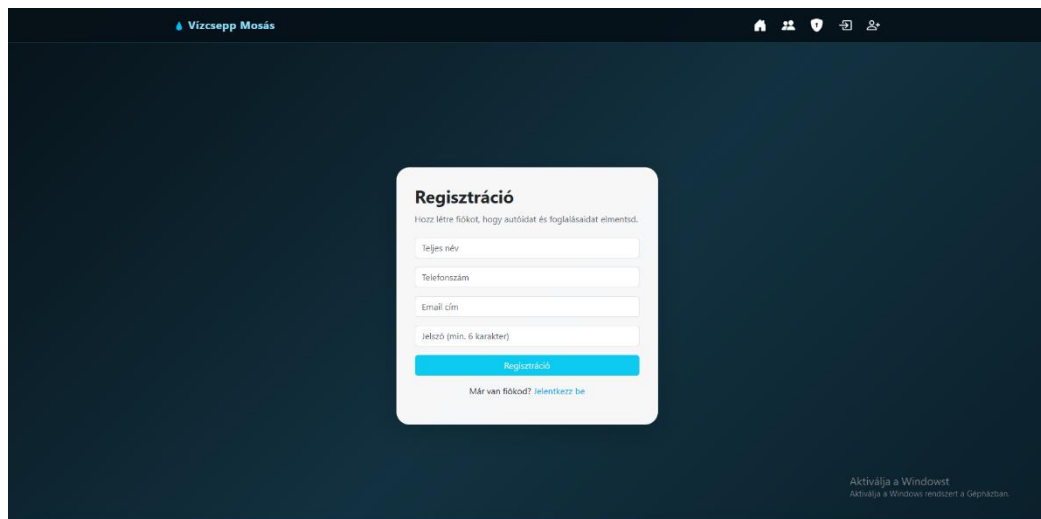
Megadandó adatok:

- teljes név
- e-mail cím
- telefonszám
- jelszó

```
register() {
  this.errorMessage = '';
  this.api.register({
    nev: this.name,
    phonenumber: this.phonenumber,
    email: this.email,
    password: this.password
  }).subscribe({
    next: (res: any) => {
      this.service.setSession(res.token, res.user);
      this.router.navigate(['/account']);
    },
    error: (err) => this.errorMessage = Object.values(err?.error?.errors ||
{}).flat().join(' ') || 'Sikertelen regisztráció.'
  });
}

navToLogin() { this.router.navigate(['/login']); }
}
```

Az űrlap Angular form validációt használ, így hibás adatok esetén a rendszer nem engedi a regisztrációt.



5. Bejelentkezési felület

A bejelentkezési oldal a felhasználók hitelesítését végzi.

Megadandó adatok:

- e-mail cím
- jelszó

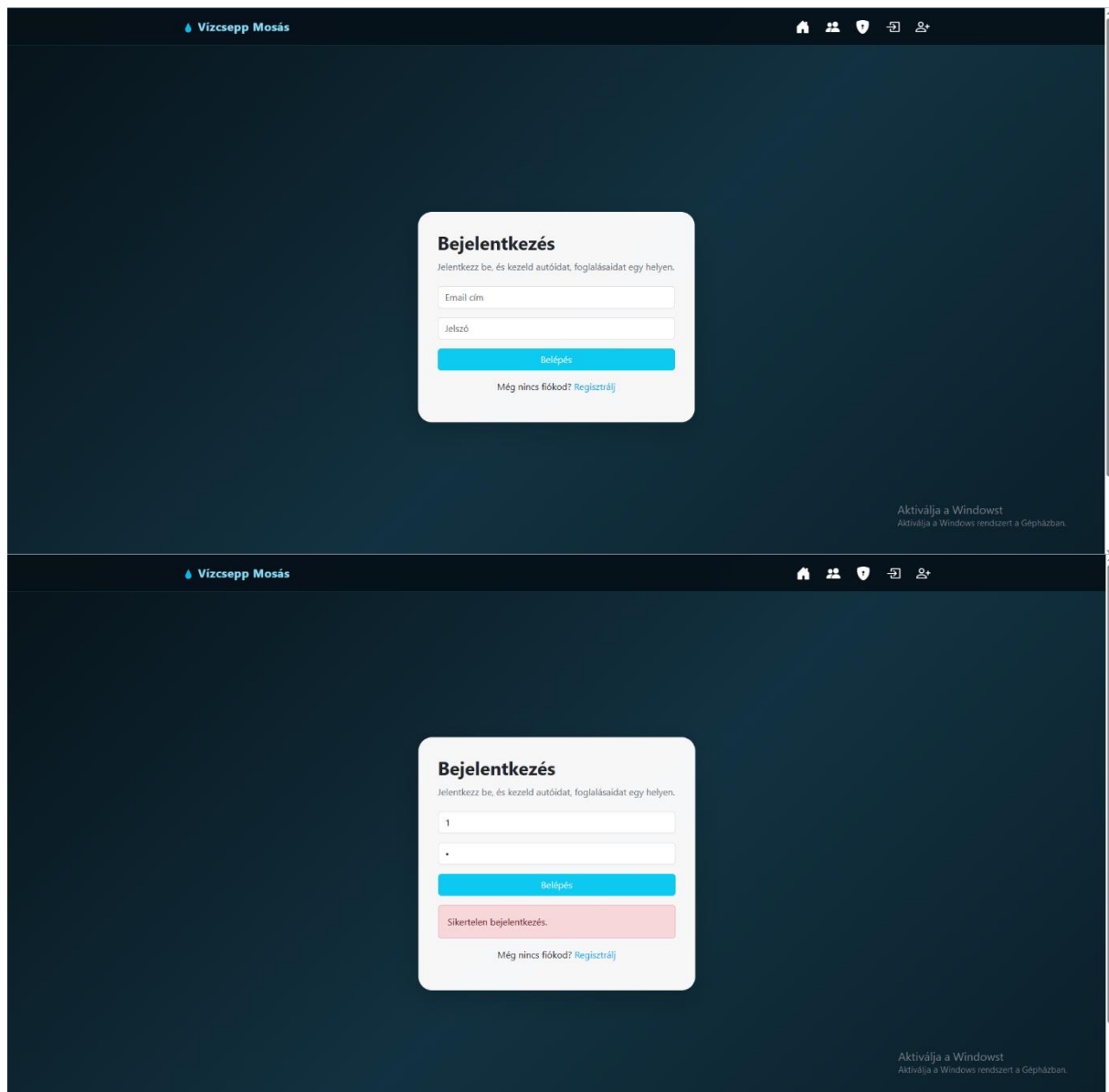
```

login() {
  this.errorMessage = '';
  this.api.login({ email: this.inputEmail, password: this.inputPassword
}).subscribe({
  next: (res: any) => {
    this.service.setSession(res.token, res.user);
    this.router.navigate(['/account']);
  },
  error: (err) => this.errorMessage = err?.error?.message || 'Sikertelen
bejelentkezés.'
  });
}

navToRegister() { this.router.navigate(['/register']); }
}

```

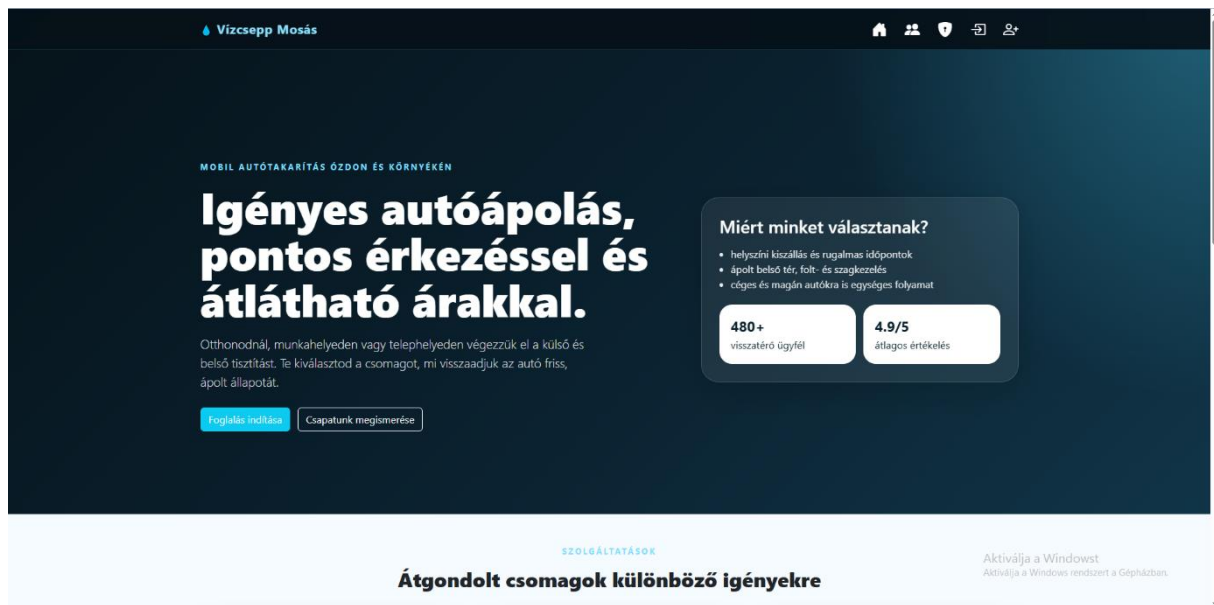
Sikeres belépés után a felhasználó hozzáfér saját fiókjához, hibás adatok esetén a rendszer nem engedi a bejelentkezést.



6. Felhasználói felület

A felület kialakításakor fontos szempont volt:

- letisztult design
- könnyű használat
- gyors navigáció
- reszponzív megjelenés



7. Angular működés a projektben

Az Angular alkalmazás moduláris felépítésű, amely komponensek segítségével kezeli az egyes funkciókat.

Előnyök:

- újrafelhasználható kód
- gyors működés
- könnyű karbantartás

8. Frontend működés

A frontend felel a felhasználói felületért és az adatok megjelenítéséért. HTTP kérések segítségével kommunikál a backenddel.

9. Komponensek bemutatása

Home

A főoldal információkat nyújt a szolgáltatásról.

Login

Felhasználói hitelesítés.

Register

Új felhasználó létrehozása.

Account

Felhasználói adatok kezelése.

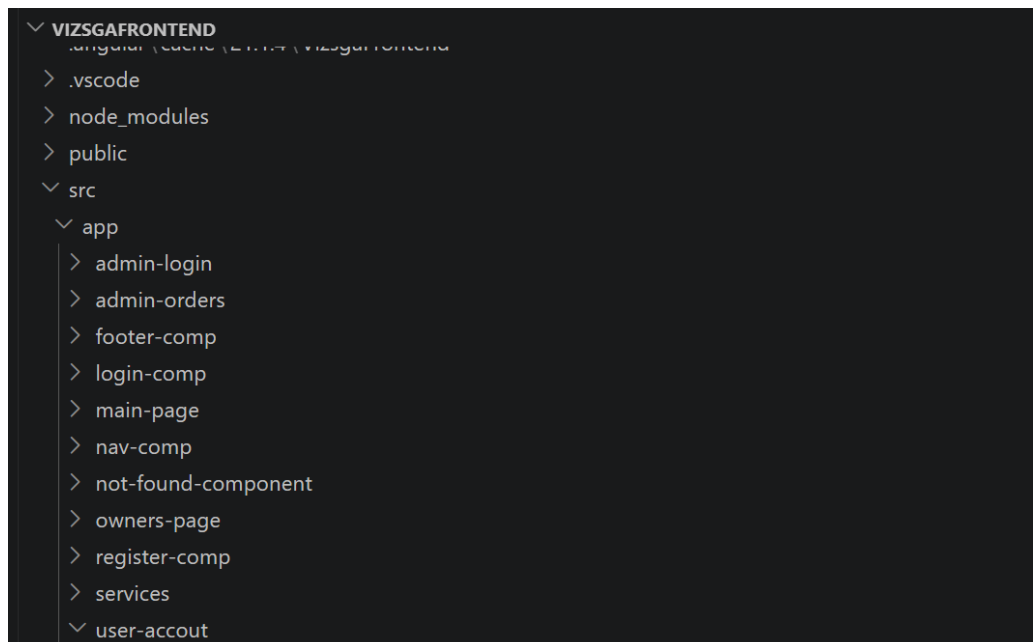
Admin

Megrendelések kezelése.

The screenshot displays the 'Vízsepp Mosás' Admin Panel. The header includes the site name and navigation icons. The main content area is titled 'ADMIN FELÜLET' and 'Beérkezett megrendelések'. It shows a summary of 1 order with a total value of 5,990 Ft. Below this is a table with the following data:

#	Ügyfél	Autó	Szolgáltatás	Dátum	Helyszín	Ár	Állapot	Művelet
2	Dániel galambosi.daniel@szikszizod.hu	Audi A4 KLF-897	Express külső mosás	2026-01-05	YX	5,990 Ft	Foglalta	Töröl

The footer contains contact information: 'Özd és környéke · helyszíni autótakarítás · info@vizseppmosas.hu', copyright '© 2026 Vízsepp Mosás', and a Windows activation notice.



10. Backend működés

A backend Laravel keretrendszerben készült, amely REST API-ként működik.

Feladatai:

- adatkezelés
- hitelesítés
- jogosultság kezelés

11. API működés

A rendszer HTTP metódusokat használ:

- GET
- POST
- PUT
- DELETE

Az adatok JSON formátumban kerülnek továbbításra.

12. Adatbázis működés

Az adatbázis relációs szerkezetű.

Táblák:

- Felhasznalo
- Auto
- Takarito
- Munka
- Szolgáltatás

Kapcsolatok idegen kulcsokkal vannak megvalósítva.

13. Jogosultságkezelés

Felhasználó:

- saját adatok kezelése

Admin:

- teljes hozzáférés
-

14. Admin felület

Az admin felület lehetővé teszi:

- megrendelések megtekintését

```
load() {  
  this.loading = true;  
  this.error = '';  
  this.api.getAdminOrders().subscribe({  
    next: (res: any) => {  
      this.orders = res;  
      this.loading = false;  
    },  
    error: (err) => {
```

```

        this.error = err?.error?.message || 'Nem sikerült lekérni a
megrendeléseket.';
        this.loading = false;
    }
});
}

```

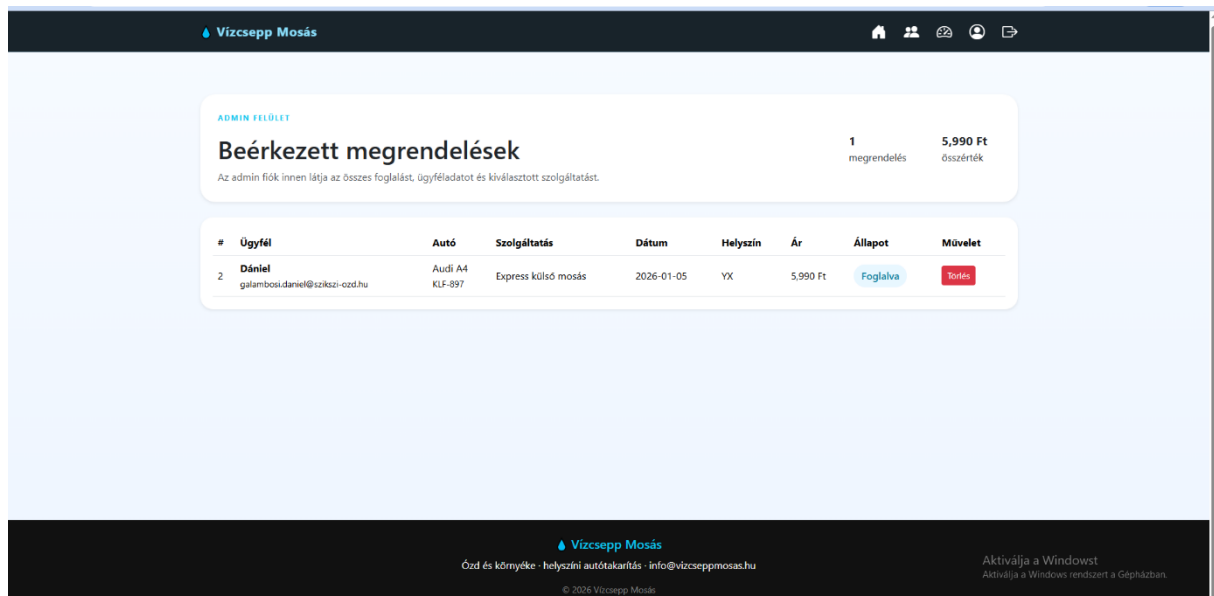
- törlés

```

deleteOrder(id: number) {
    if (!confirm('Biztosan törölni szeretnéd ezt a megrendelést?')) return;

    this.api.deleteAdminOrder(id).subscribe({
        next: () => {
            this.info = 'A megrendelés törölve lett.';
            this.orders = this.orders.filter((o) => o.id !== id);
        },
        error: (err) => {
            this.error = err?.error?.message || 'Nem sikerült törölni a
megrendelést.';
        }
    });
}

```



15. Megrendelések kezelése

A rendszer lehetőséget biztosít:

- listázásra
- törlésre
- státusz módosításra

16. Felhasználói adatok kezelése

A felhasználók kezelhetik:

- autóikat
- foglalásaikat

```

@Inject({ providedIn: 'root' })
export class ApiService {
  private baseUrl = 'http://127.0.0.1:8000/api';

  constructor(private http: HttpClient) {}

  private authOptions() {
    const token = localStorage.getItem('token') || '';
    return { headers: new HttpHeaders({ Authorization: `Bearer ${token}` }) };
  }

  login(data: any) { return this.http.post(`${this.baseUrl}/login`, data); }
  register(data: any) { return this.http.post(`${this.baseUrl}/register`,
data); }
  me() { return this.http.get(`${this.baseUrl}/me`, this.authOptions()); }
  logout() { return this.http.post(`${this.baseUrl}/logout`, {},
this.authOptions()); }
  getAutok() { return this.http.get(`${this.baseUrl}/my-autok`,
this.authOptions()); }
  addAuto(data: any) { return this.http.post(`${this.baseUrl}/my-autok`, data,
this.authOptions()); }
  deleteAuto(id: number) { return this.http.delete(`${this.baseUrl}/my-
autok/${id}`, this.authOptions()); }
  getSzolgaltatasok() { return
this.http.get(`${this.baseUrl}/szolgaltatasok`); }
  getFoglalasok() { return this.http.get(`${this.baseUrl}/my-foglalasok`,
this.authOptions()); }
  createFoglalas(data: any) { return this.http.post(`${this.baseUrl}/my-
foglalasok`, data, this.authOptions()); }
  deleteFoglalas(id: number) { return this.http.delete(`${this.baseUrl}/my-
foglalasok/${id}`, this.authOptions()); }
  getAdminOrders() { return this.http.get(`${this.baseUrl}/admin/orders`,
this.authOptions()); }
  deleteAdminOrder(id: number) { return
this.http.delete(`${this.baseUrl}/admin/orders/${id}`, this.authOptions()); }
}

```

The screenshot shows a web application for car washing services. The header is dark blue with the logo 'Vízcepp Mosás' and navigation icons. A green notification bar at the top says 'A foglalás rögzítve lett.' (The booking is confirmed).

The main content area has four panels:

- Autó mentése** (Save car): Form with fields for 'Márka' (Brand), 'Típus' (Type), 'Évjárat' (Year), 'Szín' (Color), and 'Rendszám' (License plate). A blue 'Autó mentése' button is at the bottom.
- Időpontfoglalás** (Booking): Form with dropdowns for 'Válassz autót' (Select car) and 'Válassz takarítást' (Select service). It includes a date/time picker for 'éééé. hh. nn.' and a field for 'Cím / helyszín' (Address / location). A 'Megjegyzés' (Remarks) text area and a 'Foglalás rögzítése' (Confirm booking) button are also present.
- Elmentett autók** (Saved cars): A list showing 'Audi A4' (2006, blue, KLF-897) with a 'Törles' (Delete) button.
- Korábbi foglalások** (Previous bookings): A list showing 'Express külső mosás' (2026-01-05, YX, Audi A4 - 5,990 Ft) with a 'Törles' (Delete) button.

The footer is dark blue with contact information: 'Ózd és környéke - helyszíni autótakarítás - info@vizceppmosas.hu' and a Windows activation notice.

17. Adatbiztonság

A rendszer biztosítja:

- jogosultság ellenőrzés
- adatvédelem

18. Hibakezelés

Használt státuszkódok:

- 200
- 201
- 404
- 406

19. Tesztelés

Backend:

- API tesztelés

Frontend:

- UI tesztelés
-

20. Felhasználói élmény

A cél:

- egyszerű használat
 - gyors működés
-

21. Reszponzív design

Az oldal minden eszközön működik.

22. Rendszer előnyei

- modern technológia
 - biztonság
 - gyors működés
-

23. Továbbfejlesztési lehetőségek

- email értesítés
 - fizetés
 - mobil app
-

24. Összegzés

A projekt egy modern webalkalmazás, amely valós problémára nyújt megoldást. A rendszer jól bővíthető és alkalmas további fejlesztésekre.